

INTERNAL ENGINEERING REVIEW

Autonomous Coding Agent Technical Documentation

A self-hosted, evolutionary ML research system — architecture, engineering depth, governance model, and operational risk assessment.

PREPARED FOR

**Engineering
Management**

STATUS
DATE

**June 18,
2026**

REPOSITORY

autonomous_coding_agent_advanced

**3 production
services**

**10/10 phases
implemented**

**21+ PRs
merged**

**Self-hosted LLM
stack**



Table of Contents

01 Executive Summary — what the system does, and why it matters

02 System Architecture — how the three services connect

03 The Evolutionary Engine — the core algorithm, phase by phase

04 Frontend — the operator-facing web application

05 Infrastructure / Deployment — containers, GPUs, and the build-team protocol

06 Issues Found and Fixed This Review — two live defects resolved

07 Risks, Limitations & Recommendations — five prioritized findings

08 Project Maturity Summary — where things stand, and the recommended next move

1 Executive Summary

This repository implements a **self-hosted, autonomous ML research system**: given a task definition and a dataset, it iteratively writes, runs, evaluates, and rewrites Python training scripts — using an LLM as the “researcher” — until it converges on a champion model. It is a closed-loop, evolutionary code-optimization system, not a static ML pipeline.

frontend/

Web UI to configure tasks, start/stop runs, watch live progress, and inspect the winning code. React (Vite) + nginx.

autoresearch_agent/

The evolutionary engine — generates, sandboxes, scores, and evolves candidate ML scripts. Python + FastAPI.

inference_server(s)/

Self-hosted LLM backends (no external API dependency required). llama.cpp on CUDA.

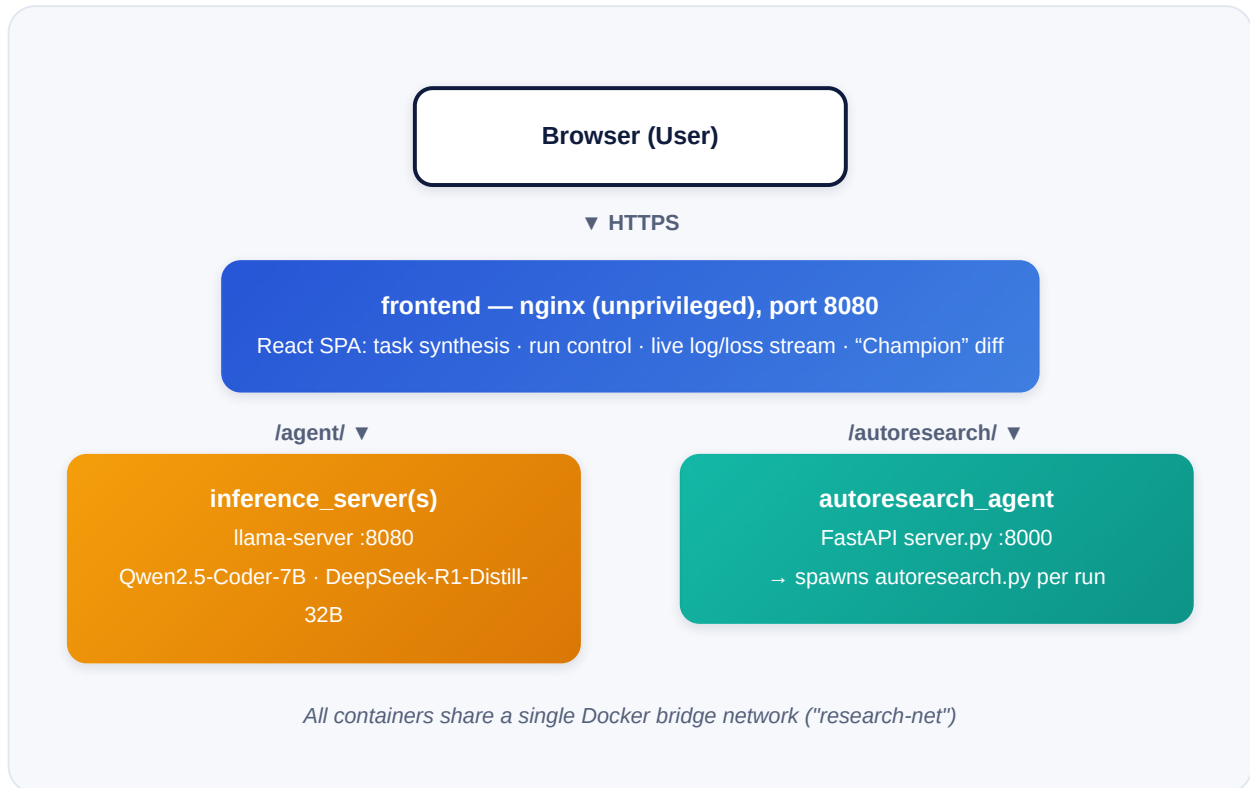
NOTABLE SIDE-FINDING

The codebase itself was developed by a small fleet of autonomous coding agents (one per service) coordinating through a written governance protocol ([AGENTS.md](#)) and an append-only audit trail ([agent_log.md](#)), rather than by a single engineer or a traditional human team. That log shows 21+ PRs merged across 11 planned phases plus several post-launch hotfixes — a useful data point on the system's traceability and maturity.

BOTTOM LINE FOR MANAGEMENT

The core algorithm is sound and reasonably sophisticated (population-based search, two-stage prompting, prompt caching, checkpoint/resume, self-healing), and has already been hardened through real production incidents (Section 6). The main open risks are operational/infra polish, not algorithmic (Section 7).

2 System Architecture



How requests flow

- The frontend never talks to the LLM or the evolutionary engine directly from the browser — nginx terminates the connection and reverse-proxies to two distinct upstream paths: `/agent/` (task synthesis via the DeepSeek backend) and `/autoresearch/` (the actual run lifecycle: `POST /run`, `POST /cancel/{run_id}`).
- `autoresearch_agent` itself talks to an LLM backend (local llama.cpp, or optionally Gemini/Anthropic if an API key is supplied) to generate and refine candidate training scripts, and to a local Python subprocess sandbox to execute them.
- The two inference containers are independent deployable units, each with their own Dockerfile, GPU offload tuning, and context window — they are **not** interchangeable without coordinated config changes (Section 5).

3 The Evolutionary Engine

`autoresearch_agent/autoresearch.py` — approximately 1,600 lines, the intellectual core of the project.

Best understood as a **genetic algorithm over source code**, where the “mutation operator” is an LLM call instead of a random bit-flip.

3.1 — Core loop, at a glance

1 Bootstrap

Establish a baseline `train.py` and its `val_loss` (auto-generated if the user didn't supply one).

2 Candidate generation

An LLM proposes N candidate modifications to the current best code, in parallel.

3 Sandboxed execution

Each candidate runs as an isolated subprocess; its `stdout` is parsed for a `val_loss` metric.

4 Selection

Candidates are scored and a new “population” is formed using softmax-weighted sampling — not greedy argmin — so the search doesn't collapse onto a local optimum too early.

5 Repeat

The loop runs for a configured number of iterations, periodically checkpointing so a crashed or interrupted run can resume exactly where it left off.

3.2 — Notable engineering decisions

CAPABILITY	MECHANISM	WHY IT MATTERS
Parallel candidates	<code>ThreadPoolExecutor</code> fans out N candidate evaluations concurrently	Without this, evaluating N ideas per generation is N× slower — directly impacts wall-clock cost per experiment.
Population selection	<code>Population</code> class + <code>select_parent</code> using Boltzmann/softmax sampling, overflow-safe normalization	Prevents premature convergence — pure “keep only the best” search gets stuck; sampling preserves exploration.

CAPABILITY	MECHANISM	WHY IT MATTERS
Two-stage prompting	A separate "Analyst" call reasons about <i>why</i> the last result changed before a "CodeGen" call writes the next candidate	Splitting diagnosis from code-writing measurably improves generation quality, at the cost of one extra LLM round-trip.
Prompt caching	Anthropic native API calls use <code>cache_control: {"type": "ephemeral"}</code> breakpoints	Cuts token cost/latency materially on long-running sessions where most of the prompt repeats every turn.
Adaptive temperature	Cosine-annealed sampling temperature across iterations	Encourages broad exploration early, convergence/exploitation late — a standard optimization heuristic applied to LLM sampling.
Checkpoint / resume	<code>save_checkpoint</code> / <code>load_checkpoint</code> persist population + experiment state to disk	A multi-hour run surviving a container restart without losing progress is an operational necessity.
History compression	<code>compress_history</code> summarizes older iterations before re-injecting them into future prompts	Keeps the prompt within the model's context window as the run grows, without losing the "lessons learned" signal.
Variance reduction	<code>robust_eval</code> re-runs near-frontier candidates multiple times and takes the median	ML training runs have run-to-run noise; without this, the search can promote a "lucky" bad candidate.
Self-healing repair loop	<code>execute_and_heal</code> catches a crashing candidate and asks the LLM to fix the traceback before discarding it	Materially increases the fraction of LLM-generated code that survives to evaluation.
Async multi-agent harness	<code>asyncio.gather</code> coordinates Director / Analyst / CodeGen / EvalWorker / SelfHealer roles via <code>asyncio.to_thread</code>	The most structurally complex phase — turns the loop into a small internal multi-role pipeline, improving throughput further.
Graceful cancellation	<code>server.py</code> 's <code>/cancel/{run_id}</code> sends SIGTERM to the run's process group, escalating to SIGKILL	Lets a user abort a long run from the UI without leaving orphaned subprocesses on the host.

3.3 — Dataset-agnostic bootstrap robustness

- **Date-format auto-detection** (day-first preferred) — avoids silent mis-parsing of ambiguous date columns.

- **Task/dataset column-name mismatch detection** — catches the common failure where a task description references a column that doesn't exist in the uploaded data, surfacing it early instead of letting the LLM hallucinate around it.
- **Authoritative column-list injection** — actual column names/dtypes are injected verbatim into prompts rather than relying on the LLM to infer them.

3.4 — The `val_loss` abstraction

`val_loss` is a deliberately generic, LLM-chosen, "lower is better" fitness signal, extracted from candidate stdout via regex — not hardcoded to a specific metric like MSE. In practice, for regression tasks the LLM typically does emit MSE (confirmed during this review by inspecting a live run: a `GradientBoostingRegressor` baseline reported `val_loss=55.68`, the literal MSE on that column's scale, not a normalized 0–1 score).

WORTH STATING PLAINLY TO MANAGEMENT

MSE is not a percentage and is not bounded to [0,1] — it scales with the variance of the target variable, so absolute `val_loss` numbers are only meaningful relative to other runs on the *same* dataset/target, not as a universal quality bar.

3.5 — `server.py` (131 lines)

A thin FastAPI wrapper around the engine: `POST /run` accepts task/baseline/iterations/model-choice/API-key/optional dataset as multipart form data, creates a per-request working directory, launches the engine as a subprocess, and streams stdout back to the client (final code wrapped in sentinel markers). `POST /cancel/{run_id}` terminates that run's process group.

4 Frontend

`frontend/src/App.jsx` — approximately 1,490 lines.

A single-page React app (Vite build, served by nginx) with no separate backend of its own — it is purely a client for the two proxied services described in Section 2.

Task Synthesiser

LLM-assisted flow that turns an informal description into a structured task definition, via the DeepSeek backend at `/agent/`.

Run Tabs & Bootstrap Modal

Generate / Task / Baseline / Config tabs, plus a Baseline Bootstrap modal for triggering and inspecting the auto-generated baseline.

Streaming Log Parser

Buffers and parses the raw HTTP stream line-by-line: plain text, `__RUN_ID__:<id>`, and `__EVENT__{json}` structured events.

Live Loss Sparkline

An inline SVG chart driven by parsed loss events — a real-time visual of convergence during a run.

“Champion” Diff Tab

Client-side Myers diff renders a green/red line diff between baseline and final winning code.

Run Control

`startEvolution` posts to `/autoresearch/run` ;
`handleStop` posts to `/autoresearch/cancel/{id}` .

5 Infrastructure / Deployment

5.1 — Container topology

CONTAINER	BASE IMAGE	NON-ROOT USER	PORT	PURPOSE
frontend	nginx-unprivileged 1.25-alpine-slim (multi-stage from node:20-alpine)	UID 101	8080	Static SPA + reverse proxy
autoresearch_agent (ai-researcher)	UBI9 → UBI9-minimal	researcher:10002	8000	Evolution engine + FastAPI
inference_server (Qwen)	nvidia/cuda 12.2.2 devel/runtime-ubi9	llmuser:10001	8080	Code-gen LLM — -c 16384 -nvl 99 -- parallel 4 --mlock
inference_server_reasoning (local-deepseek-backend)	same CUDA base	llmuser:10001	8080	Reasoning LLM — -c 8192 -nvl 61 -fa on - np 1 --no- warmup

All four are wired together by a shared Docker bridge network (`research-net`) and each built with multi-stage Dockerfiles and non-root execution — a sensible security baseline (no container runs as root in its final stage).

5.2 — GPU footprint

The DeepSeek-R1-Distill-Qwen-32B Q3_K_M model is ~14.8 GB on disk. The Dockerfile comment claims it's "guaranteed to fit 100% in your 16GB VRAM," but live `nvidia-smi` inspection during this engagement showed the actual card is a **24GB Quadro RTX 6000** — the in-repo comment is stale and should be corrected to avoid misleading future infra decisions. The current entrypoint partially offloads (-

ngl 61 , deliberately spilling some layers to CPU) — that headroom is what allowed today's context-window fix to land safely.

5.3 — Multi-agent software development governance

This repository was built under a written multi-agent protocol (`AGENTS.md` , `agent_log.md` , per-directory `CLAUDE.md` files) — worth highlighting because it explains *why* the codebase is structured the way it is.

- Each service directory has exactly one owning “agent” (`ara` = autoresearch, `fe` = frontend, `inf` = inference server), with hard rules against editing outside one's own directory without a Director-approved log entry.
- Branches follow `agent/<abbrev>/phase-<N>-<slug>` ; one phase per branch, one branch per PR (capped at 400 changed lines), and **no agent may merge its own PR**.
- A merge-dependency graph governs sequencing (e.g., Phase 2's population logic is blocked until Phase 1's parallel-candidate plumbing merges).
- `agent_log.md` is an append-only audit trail: every branch creation, PR open, rebase, and merge is logged before the action is taken, including explicit `BLOCKED:<agent>` markers.

TAKEAWAY FOR MANAGEMENT

This isn't an ungoverned pile of agent output — there is a real (if informal) process model with traceability, and the log shows it being followed in practice across 21+ logged PR-open events and corresponding merges.

6 Issues Found and Fixed This Review

Two live production defects were diagnosed and fixed as part of this engagement, both now committed:

- 1 nginx upstream resolution failure (frontend/nginx.conf)**

The `/agent/` proxy pointed at a hostname (`local-qwen-backend`) that didn't match the container actually running on the shared network (`local-deepseek-backend`), causing `nginx: [emerg] host not found in upstream` . Fixed by correcting the `proxy_pass` target.
- 2 LLM context-window overflow (inference_server_reasoning/Dockerfile)**

The DeepSeek backend was configured with `-c 4096` , but the bootstrap prompt the engine sends is consistently ~5,600+ tokens, producing an HTTP 400 ("request exceeds context size"). Root-caused via Dockerfile inspection, then verified against live `nvidia-smi` VRAM headroom (~7.6 GB free) before bumping context to `-c 8192` , confirmed to fit with margin.

WHY THIS IS INCLUDED HERE

These are recorded primarily as evidence of the system's current operational maturity: the architecture is sound, but it has been under active, hands-on tuning right up to recent runs — not a finished, "set and forget" product yet.

7 Risks, Limitations & Recommendations

#	SEVERITY	FINDING	WHY IT MATTERS	RECOMMENDATION
1	High	<code>autoresearch_agent/Dockerfile</code> clones <code>github.com/karpathy/autoresearch.git</code> at build time, then immediately overwrites the two files that matter with the repo's own versions.	Vestigial clone step pulls an unrelated, unpinned upstream repo whose only surviving artifact is a <code>prepare.py</code> the build <code>chmod</code> s but never calls — a supply-chain and build-fragility risk for zero functional benefit.	Remove the <code>git clone</code> step; inline only the actually-needed requirements and drop unused heavy deps (<code>torch</code> , <code>sdv</code> , <code>copulas</code> , <code>ctgan</code> , <code>imbalanced-learn</code>) — shrinks the image and the vulnerability surface.
2	Medium	Stale hardware assumption in <code>inference_server_reasoning/Dockerfile</code> comments (claims 16GB VRAM; actual card is 24GB).	Future tuning decisions made by reading the comment instead of <code>nvidia-smi</code> could under-utilize available GPU memory, or be misapplied to a genuinely 16GB deployment.	Correct the comment to reflect measured VRAM; prefer probing <code>nvidia-smi</code> at startup over hardcoded assumptions if targeting heterogeneous hardware.
3	Medium	Two independent LLM backends each hand-tuned for context/offload, with no automated check they stay within combined VRAM budget.	The Section 6 incident happened because a config change	Add a lightweight CI/build-time check (or documented VRAM budget table) totaling expected

#	SEVERITY	FINDING	WHY IT MATTERS	RECOMMENDATION
			in one container wasn't validated against the other's footprint until it broke in production.	usage of both backends running concurrently.
4	Low	<code>agent_log.md</code> -driven governance is manual and convention-based, not tool-enforced.	Works today, but no hard guarantee against a future agent run skipping the log step under context pressure or interruption.	If this pattern scales to more agents, add a pre-commit/pre-push hook that mechanically blocks a push without a matching log entry.
5	Low	<code>val_loss</code> is a free-form, LLM-emitted number with no enforced units or cross-run normalization.	Cross-run/cross-dataset "val_loss = X" comparisons are not meaningful without knowing the metric and target scale that produced them.	If used for cross-task leaderboarding, standardize on a normalized metric (e.g., R^2 or normalized RMSE) while keeping raw val_loss for within-task optimization.

8 Project Maturity Summary

- **Algorithmic core:** all 10 originally planned engine phases are implemented and merged, per both direct code inspection and the `agent_log.md` audit trail.
- **Post-launch hardening:** at least five hotfixes have already shipped after initial launch — a `select_parent` numeric-overflow fix, baseline-bootstrap gating, bootstrap date-format detection, a stream-interleaving fix, and task/dataset column-mismatch detection.
- **This engagement** added two more real fixes (Section 6) and surfaced five further findings (Section 7) recommended for the next iteration.

Recommendation for management

The system is functionally complete against its original design and already self-correcting in response to production issues. The highest-leverage next investments are the Dockerfile/dependency cleanup (finding 1) and a VRAM/config budget check (finding 3) — both small, contained changes that materially reduce operational surprise risk without touching the core algorithm.